Summary 374
Exercises 375
References 376

Chapter 9 Graph Algorithms 379

9.1 Definitions 379
9.1.1 Representation of Graphs 380
9.2 Topological Sort 382
9.3 Shortest-Path Algorithms 386
9.3.1 Unweighted Shortest Paths 387
9.3.2 Dijkstra's Algorithm 391
9.3.3 Graphs with Negative Edge Costs 400
9.3.4 Acyclic Graphs 400
9.3.5 All-Pairs Shortest Path 404
9.3.6 Shortest Path Example 404
9.4 Network Flow Problems 406
9.4.1 A Simple Maximum-Flow Algorithm 408
9.5 Minimum Spanning Tree 413
9.5.1 Prim's Algorithm 414
9.5.2 Kruskal's Algorithm 417
9.6 Applications of Depth-First Search 419
9.6.1 Undirected Graphs 420
9.6.2 Biconnectivity 421
9.6.3 Euler Circuits 425
9.6.4 Directed Graphs 429
9.6.5 Finding Strong Components 431
9.7 Introduction to NP-Completeness 432
9.7.1 Easy vs. Hard 433
9.7.2 The Class NP 434
9.7.3 NP-Complete Problems 434
Summary 437
Exercises 437
References 445

Chapter 10 Algorithm Design Techniques 449

10.1 Greedy Algorithms 449
10.1.1 A Simple Scheduling Problem 450
10.1.2 Huffman Codes 453
10.1.3 Approximate Bin Packing 459
10.2 Divide and Conquer 467
10.2.1 Running Time of Divide-and-Conquer Algorithms 468
10.2.2 Closest-Points Problem 470

10.2.3 The Selection Problem 475
10.2.4 Theoretical Improvements for Arithmetic Problems 478
10.3 Dynamic Programming 482
10.3.1 Using a Table Instead of Recursion 483
10.3.2 Ordering Matrix Multiplications 485
10.3.3 Optimal Binary Search Tree 487
10.3.4 All-Pairs Shortest Path 491
10.4 Randomized Algorithms 494
10.4.1 Random-Number Generators 495
10.4.2 Skip Lists 500
10.4.3 Primality Testing 503
10.5 Backtracking Algorithms 506
10.5.1 The Turnpike Reconstruction Problem 506
10.5.2 Games 511
Summary 518
Exercises 518
References 527

Chapter 11 Amortized Analysis 533

11.1 An Unrelated Puzzle 534
11.2 Binomial Queues 534
11.3 Skew Heaps 539
11.4 Fibonacci Heaps 541
11.4.1 Cutting Nodes in Leftist Heaps 542
11.4.2 Lazy Merging for Binomial Queues 544
11.4.3 The Fibonacci Heap Operations 548
11.4.4 Proof of the Time Bound 549
11.5 Splay Trees 551
Summary 555
Exercises 556
References 557

Chapter 12 Advanced Data Structures and Implementation 559

12.1 Top-Down Splay Trees 559
12.2 Red-Black Trees 566
12.2.1 Bottom-Up Insertion 567
12.2.2 Top-Down Red-Black Trees 568
12.2.3 Top-Down Deletion 570
12.3 Treaps 576

12.4 Suffix Arrays and Suffix Trees 579
12.4.1 Suffix Arrays 580
12.4.2 Suffix Trees 583
12.4.3 Linear-Time Construction of Suffix Arrays and Suffix Trees 586
12.5 *k*-d Trees 596
12.6 Pairing Heaps 602
Summary 606
Exercises 608
References 612

Appendix A Separate Compilation of Class Templates 615

A.1 Everything in the Header 616
A.2 Explicit Instantiation 616

Index 619

PREFACE

Purpose/Goals

The fourth edition of *Data Structures and Algorithm Analysis in C++* describes *data structures,* methods of organizing large amounts of data, and *algorithm analysis,* the estimation of the running time of algorithms. As computers become faster and faster, the need for programs that can handle large amounts of input becomes more acute. Paradoxically, this requires more careful attention to efficiency, since inefficiencies in programs become most obvious when input sizes are large. By analyzing an algorithm before it is actually coded, students can decide if a particular solution will be feasible. For example, in this text students look at specific problems and see how careful implementations can reduce the time constraint for large amounts of data from centuries to less than a second. Therefore, no algorithm or data structure is presented without an explanation of its running time. In some cases, minute details that affect the running time of the implementation are explored.

Once a solution method is determined, a program must still be written. As computers have become more powerful, the problems they must solve have become larger and more complex, requiring development of more intricate programs. The goal of this text is to teach students good programming and algorithm analysis skills simultaneously so that they can develop such programs with the maximum amount of efficiency.

This book is suitable for either an advanced data structures course or a first-year graduate course in algorithm analysis. Students should have some knowledge of intermediate programming, including such topics as pointers, recursion, and object-based programming, as well as some background in discrete math.

Approach

Although the material in this text is largely language-independent, programming requires the use of a specific language. As the title implies, we have chosen C++ for this book.

C++ has become a leading systems programming language. In addition to fixing many of the syntactic flaws of C, C++ provides direct constructs (the *class* and *template*) to implement generic data structures as abstract data types.

The most difficult part of writing this book was deciding on the amount of C++ to include. Use too many features of C++ and one gets an incomprehensible text; use too few and you have little more than a C text that supports classes.

The approach we take is to present the material in an *object-based approach.* As such, there is almost no use of inheritance in the text. We use class templates to describe generic data structures. We generally avoid esoteric C++ features and use the `vector` and `string` classes that are now part of the C++ standard. Previous editions have implemented class templates by separating the class template interface from its implementation. Although this is arguably the preferred approach, it exposes compiler problems that have made it

difficult for readers to actually use the code. As a result, in this edition the online code represents class templates as a single unit, with no separation of interface and implementation. Chapter 1 provides a review of the C++ features that are used throughout the text and describes our approach to class templates. Appendix A describes how the class templates could be rewritten to use separate compilation.

Complete versions of the data structures, in both C++ and Java, are available on the Internet. We use similar coding conventions to make the parallels between the two languages more evident.

Summary of the Most Significant Changes in the Fourth Edition

The fourth edition incorporates numerous bug fixes, and many parts of the book have undergone revision to increase the clarity of presentation. In addition,

- Chapter 4 includes implementation of the AVL tree deletion algorithm—a topic often requested by readers.
- Chapter 5 has been extensively revised and enlarged and now contains material on two newer algorithms: cuckoo hashing and hopscotch hashing. Additionally, a new section on universal hashing has been added. Also new is a brief discussion of the `unordered_set` and `unordered_map` class templates introduced in C++11.
- Chapter 6 is mostly unchanged; however, the implementation of the binary heap makes use of move operations that were introduced in C++11.
- Chapter 7 now contains material on radix sort, and a new section on lower-bound proofs has been added. Sorting code makes use of move operations that were introduced in C++11.
- Chapter 8 uses the new union/find analysis by Seidel and Sharir and shows the $O(M\,\alpha(M,N))$ bound instead of the weaker $O(M \log^* N)$ bound in prior editions.
- Chapter 12 adds material on suffix trees and suffix arrays, including the linear-time suffix array construction algorithm by Karkkainen and Sanders (with implementation). The sections covering deterministic skip lists and AA-trees have been removed.
- Throughout the text, the code has been updated to use C++11. Notably, this means use of the new C++11 features, including the `auto` keyword, the range `for` loop, move construction and assignment, and uniform initialization.

Overview

Chapter 1 contains review material on discrete math and recursion. I believe the only way to be comfortable with recursion is to see good uses over and over. Therefore, recursion is prevalent in this text, with examples in every chapter except Chapter 5. Chapter 1 also includes material that serves as a review of basic C++. Included is a discussion of templates and important constructs in C++ class design.

Chapter 2 deals with algorithm analysis. This chapter explains asymptotic analysis and its major weaknesses. Many examples are provided, including an in-depth explanation of logarithmic running time. Simple recursive programs are analyzed by intuitively converting them into iterative programs. More complicated divide-and-conquer programs are introduced, but some of the analysis (solving recurrence relations) is implicitly delayed until Chapter 7, where it is performed in detail.

Chapter 3 covers lists, stacks, and queues. This chapter includes a discussion of the STL `vector` and `list` classes, including material on iterators, and it provides implementations of a significant subset of the `STL vector and list` classes.

Chapter 4 covers trees, with an emphasis on search trees, including external search trees (B-trees). The UNIX file system and expression trees are used as examples. AVL trees and splay trees are introduced. More careful treatment of search tree implementation details is found in Chapter 12. Additional coverage of trees, such as file compression and game trees, is deferred until Chapter 10. Data structures for an external medium are considered as the final topic in several chapters. Included is a discussion of the STL `set` and `map` classes, including a significant example that illustrates the use of three separate maps to efficiently solve a problem.

Chapter 5 discusses hash tables, including the classic algorithms such as separate chaining and linear and quadratic probing, as well as several newer algorithms, namely cuckoo hashing and hopscotch hashing. Universal hashing is also discussed, and extendible hashing is covered at the end of the chapter.

Chapter 6 is about priority queues. Binary heaps are covered, and there is additional material on some of the theoretically interesting implementations of priority queues. The Fibonacci heap is discussed in Chapter 11, and the pairing heap is discussed in Chapter 12.

Chapter 7 covers sorting. It is very specific with respect to coding details and analysis. All the important general-purpose sorting algorithms are covered and compared. Four algorithms are analyzed in detail: insertion sort, Shellsort, heapsort, and quicksort. New to this edition is radix sort and lower bound proofs for selection-related problems. External sorting is covered at the end of the chapter.

Chapter 8 discusses the disjoint set algorithm with proof of the running time. This is a short and specific chapter that can be skipped if Kruskal's algorithm is not discussed.

Chapter 9 covers graph algorithms. Algorithms on graphs are interesting, not only because they frequently occur in practice but also because their running time is so heavily dependent on the proper use of data structures. Virtually all of the standard algorithms are presented along with appropriate data structures, pseudocode, and analysis of running time. To place these problems in a proper context, a short discussion on complexity theory (including *NP*-completeness and undecidability) is provided.

Chapter 10 covers algorithm design by examining common problem-solving techniques. This chapter is heavily fortified with examples. Pseudocode is used in these later chapters so that the student's appreciation of an example algorithm is not obscured by implementation details.

Chapter 11 deals with amortized analysis. Three data structures from Chapters 4 and 6 and the Fibonacci heap, introduced in this chapter, are analyzed.

Chapter 12 covers search tree algorithms, the suffix tree and array, the *k*-d tree, and the pairing heap. This chapter departs from the rest of the text by providing complete and careful implementations for the search trees and pairing heap. The material is structured so that the instructor can integrate sections into discussions from other chapters. For example, the top-down red-black tree in Chapter 12 can be discussed along with AVL trees (in Chapter 4).

Chapters 1 to 9 provide enough material for most one-semester data structures courses. If time permits, then Chapter 10 can be covered. A graduate course on algorithm analysis could cover chapters 7 to 11. The advanced data structures analyzed in Chapter 11 can easily be referred to in the earlier chapters. The discussion of *NP*-completeness in Chapter 9

is far too brief to be used in such a course. You might find it useful to use an additional work on *NP*-completeness to augment this text.

Exercises

Exercises, provided at the end of each chapter, match the order in which material is presented. The last exercises may address the chapter as a whole rather than a specific section. Difficult exercises are marked with an asterisk, and more challenging exercises have two asterisks.

References

References are placed at the end of each chapter. Generally the references either are historical, representing the original source of the material, or they represent extensions and improvements to the results given in the text. Some references represent solutions to exercises.

Supplements

The following supplements are available to all readers at http://cssupport.pearsoncmg.com/

- Source code for example programs
- Errata

In addition, the following material is available only to qualified instructors at Pearson Instructor Resource Center (www.pearsonhighered.com/irc). Visit the IRC or contact your Pearson Education sales representative for access.

- Solutions to selected exercises
- Figures from the book
- Errata

Acknowledgments

Many, many people have helped me in the preparation of books in this series. Some are listed in other versions of the book; thanks to all.

As usual, the writing process was made easier by the professionals at Pearson. I'd like to thank my editor, Tracy Johnson, and production editor, Marilyn Lloyd. My wonderful wife Jill deserves extra special thanks for everything she does.

Finally, I'd like to thank the numerous readers who have sent e-mail messages and pointed out errors or inconsistencies in earlier versions. My website www.cis.fiu.edu/~weiss will also contain updated source code (in C++ and Java), an errata list, and a link to submit bug reports.

M.A.W.
Miami, Florida

CHAPTER 1

Programming: A General Overview

In this chapter, we discuss the aims and goals of this text and briefly review programming concepts and discrete mathematics. We will . . .

- See that how a program performs for reasonably large input is just as important as its performance on moderate amounts of input.
- Summarize the basic mathematical background needed for the rest of the book.
- Briefly review **recursion**.
- Summarize some important features of C++ that are used throughout the text.

1.1 What's This Book About?

Suppose you have a group of N numbers and would like to determine the kth largest. This is known as the **selection problem**. Most students who have had a programming course or two would have no difficulty writing a program to solve this problem. There are quite a few "obvious" solutions.

One way to solve this problem would be to read the N numbers into an array, sort the array in decreasing order by some simple algorithm such as bubble sort, and then return the element in position k.

A somewhat better algorithm might be to read the first k elements into an array and sort them (in decreasing order). Next, each remaining element is read one by one. As a new element arrives, it is ignored if it is smaller than the kth element in the array. Otherwise, it is placed in its correct spot in the array, bumping one element out of the array. When the algorithm ends, the element in the kth position is returned as the answer.

Both algorithms are simple to code, and you are encouraged to do so. The natural questions, then, are: Which algorithm is better? And, more important, Is either algorithm good enough? A simulation using a random file of 30 million elements and $k = 15{,}000{,}000$ will show that neither algorithm finishes in a reasonable amount of time; each requires several days of computer processing to terminate (albeit eventually with a correct answer). An alternative method, discussed in Chapter 7, gives a solution in about a second. Thus, although our proposed algorithms work, they cannot be considered good algorithms,

	1	2	3	4
1	t	h	i	s
2	w	a	t	s
3	o	a	h	g
4	f	g	d	t

Figure 1.1 Sample word puzzle

because they are entirely impractical for input sizes that a third algorithm can handle in a reasonable amount of time.

A second problem is to solve a popular word puzzle. The input consists of a two-dimensional array of letters and a list of words. The object is to find the words in the puzzle. These words may be horizontal, vertical, or diagonal in any direction. As an example, the puzzle shown in Figure 1.1 contains the words *this, two, fat,* and *that.* The word *this* begins at row 1, column 1, or (1,1), and extends to (1,4); *two* goes from (1,1) to (3,1); *fat* goes from (4,1) to (2,3); and *that* goes from (4,4) to (1,1).

Again, there are at least two straightforward algorithms that solve the problem. For each word in the word list, we check each ordered triple (*row, column, orientation*) for the presence of the word. This amounts to lots of nested `for` loops but is basically straightforward.

Alternatively, for each ordered quadruple (*row, column, orientation, number of characters*) that doesn't run off an end of the puzzle, we can test whether the word indicated is in the word list. Again, this amounts to lots of nested `for` loops. It is possible to save some time if the maximum number of characters in any word is known.

It is relatively easy to code up either method of solution and solve many of the real-life puzzles commonly published in magazines. These typically have 16 rows, 16 columns, and 40 or so words. Suppose, however, we consider the variation where only the puzzle board is given and the word list is essentially an English dictionary. Both of the solutions proposed require considerable time to solve this problem and therefore might not be acceptable. However, it is possible, even with a large word list, to solve the problem very quickly.

An important concept is that, in many problems, writing a working program is not good enough. If the program is to be run on a large data set, then the running time becomes an issue. Throughout this book we will see how to estimate the running time of a program for large inputs and, more important, how to compare the running times of two programs without actually coding them. We will see techniques for drastically improving the speed of a program and for determining program bottlenecks. These techniques will enable us to find the section of the code on which to concentrate our optimization efforts.

1.2 Mathematics Review

This section lists some of the basic formulas you need to memorize, or be able to derive, and reviews basic proof techniques.

1.2.1 Exponents

$$X^A X^B = X^{A+B}$$
$$\frac{X^A}{X^B} = X^{A-B}$$
$$(X^A)^B = X^{AB}$$
$$X^N + X^N = 2X^N \neq X^{2N}$$
$$2^N + 2^N = 2^{N+1}$$

1.2.2 Logarithms

In computer science, all logarithms are to the base 2 unless specified otherwise.

Definition 1.1
$X^A = B$ if and only if $\log_X B = A$

Several convenient equalities follow from this definition.

Theorem 1.1

$$\log_A B = \frac{\log_C B}{\log_C A}; \quad A, B, C > 0, A \neq 1$$

Proof
Let $X = \log_C B$, $Y = \log_C A$, and $Z = \log_A B$. Then, by the definition of logarithms, $C^X = B$, $C^Y = A$, and $A^Z = B$. Combining these three equalities yields $B = C^X = (C^Y)^Z$. Therefore, $X = YZ$, which implies $Z = X/Y$, proving the theorem.

Theorem 1.2

$$\log AB = \log A + \log B; \quad A, B > 0$$

Proof
Let $X = \log A$, $Y = \log B$, and $Z = \log AB$. Then, assuming the default base of 2, $2^X = A$, $2^Y = B$, and $2^Z = AB$. Combining the last three equalities yields $2^X 2^Y = AB = 2^Z$. Therefore, $X + Y = Z$, which proves the theorem.

Some other useful formulas, which can all be derived in a similar manner, follow.

$$\log A/B = \log A - \log B$$

$$\log(A^B) = B \log A$$

$$\log X < X \quad \text{for all } X > 0$$

$$\log 1 = 0, \qquad \log 2 = 1, \qquad \log 1{,}024 = 10, \qquad \log 1{,}048{,}576 = 20$$

1.2.3 Series

The easiest formulas to remember are

$$\sum_{i=0}^{N} 2^i = 2^{N+1} - 1$$

and the companion,

$$\sum_{i=0}^{N} A^i = \frac{A^{N+1} - 1}{A - 1}$$

In the latter formula, if $0 < A < 1$, then

$$\sum_{i=0}^{N} A^i \leq \frac{1}{1 - A}$$

and as N tends to ∞, the sum approaches $1/(1 - A)$. These are the "geometric series" formulas.

We can derive the last formula for $\sum_{i=0}^{\infty} A^i$ $(0 < A < 1)$ in the following manner. Let S be the sum. Then

$$S = 1 + A + A^2 + A^3 + A^4 + A^5 + \cdots$$

Then

$$AS = A + A^2 + A^3 + A^4 + A^5 + \cdots$$

If we subtract these two equations (which is permissible only for a convergent series), virtually all the terms on the right side cancel, leaving

$$S - AS = 1$$

which implies that

$$S = \frac{1}{1 - A}$$

We can use this same technique to compute $\sum_{i=1}^{\infty} i/2^i$, a sum that occurs frequently. We write

$$S = \frac{1}{2} + \frac{2}{2^2} + \frac{3}{2^3} + \frac{4}{2^4} + \frac{5}{2^5} + \cdots$$

and multiply by 2, obtaining

$$2S = 1 + \frac{2}{2} + \frac{3}{2^2} + \frac{4}{2^3} + \frac{5}{2^4} + \frac{6}{2^5} + \cdots$$

Subtracting these two equations yields

$$S = 1 + \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} + \cdots$$

Thus, $S = 2$.